DJANGO REST FRAMEWORK (DRF)

WHAT IS REST?

- **RE**presentational **S**tate **T**ransfer (REST) is an architectural style that defines a set of constraints to be used for creating web services.
- REST API is a way of accessing web services in a simple and flexible way without having any processing.
- It's used to fetch or give some information from a web service. All communication done via REST API uses only HTTP request.

WHY REST?

- **Lightweight** - One of the main benefits of REST APIs is that they rely on the HTTP standard, which means it's format-agonistic and you can use XML, JSON, HTML, etc. This makes REST APIs fast, and lightweight — which is necessary for mobile app projects, internet of things devices, and more.
- **Independent** - Another benefit of REST APIs is the fact that the client and server are independent. In other words, the REST protocol separates the data storage and the UI from the server. This means that developers can work on different areas of a project independently and try out multiple developer environments as needed.
- **Scalable and flexible** - One of the third benefits of REST APIs, and perhaps one of the most important, is scalability and flexibility . REST APIs can be scaled quickly primarily due to the separation between the client and the server. Additionally, developers can also easily integrate REST APIs without much added work.

SOFTWARE SETUP

- Install Python, Django, Django Rest Framework.
- A database like PostgreSQL and Database software like Valentina Studio.

FUNCTION BASED VIEW

1. **Create a Project –**
   a. Run this command on terminal "**django-admin startproject fbvSerializers**".
   b. Change directory to that folder "**cd fbvSerializers**" and then use command "**py manage.py startapp fbvApp**".
   c. Open fbvSerializers in VS Code and open settings.py of project fbvSerializers then configure fbvapp and rest framework there.

```
INSTALLED_APPS = [
    'fbvApp',
    'rest_framework'
]
```

   d. Then scroll down and configure database.

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'studentdb',
        'USER': 'postgres',
        'PASSWORD': 'root',
        'HOST': 'localhost'
    }
}
```

2. **Create the Model –**
   a. Open fbvApp models.py and the code should be like this.

```
from django.db import models

# Create your models here.
class Student(models.Model):
    id = models.IntegerField(primary_key=True)
    name = models.CharField(max_length=20)
    score = models.DecimalField(max_digits=10, decimal_places=3)

    def __str__(self):
        return self.id+self.name+self.score
```

3. **Migration –**
   a. Before doing the migration, we have to create the database **studentdb** which we mentioned in the database section of settings.py.
   b. Open Valentina Studio and create the database named **studentdb**.
   c. Use command "**py manage.py makemigrations**" and then "**py manage.py migrate**".

4. **Serializers –**
   a. Goto fbvApp and create a file named serializers.py.
   b. Open it and the code should be this.

```
from attr import field
from rest_framework import serializers
from fbvApp.models import Student

class StudentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Student
        fields = ['id', 'name', 'score']
```

5. **Views.py of fbvApp (Defining functions) –**

a. Open **views.py** of fbvApp and define the student_list function for GET and POST method, the code should be like this.

```
i.   from django.shortcuts import render
ii.  from fbvApp.models import Student
iii. from fbvApp.serializers import StudentSerializer
iv.  from rest_framework.response import Response
v.   from rest_framework import status
vi.  from rest_framework.decorators import api_view
vii.
viii. # Create your views here.
ix.  @api_view(['GET', 'POST'])
x.   def student_list(request):
xi.      if request.method == 'GET':
xii.         students = Student.objects.all()
xiii.        serializer = StudentSerializer(students, many=True)
xiv.         return Response(serializer.data)
xv.
xvi.     elif request.method == 'POST':
xvii.            serializer = StudentSerializer(data =  request.data)
xviii.           if serializer.is_valid():
xix.                 serializer.save()
xx.                  return Response(serializer.data, status =
     status.HTTP_201_CREATED)
xxi.             return Response(serializer.errors, status =
     status.HTTP_400_BAD_REQUEST)
```

b. Now to implement primary key based operations we will define student_details function in **views.py** of fbvApp for GET, PUT and DELETE method, the code should be like this.

```
i.   @api_view(['GET', 'PUT', 'DELETE'])
ii.  def student_detail(request, pk):
iii.     try:
iv.          student = Student.objects.get(pk=pk)
v.       except Student.DoesNotExist:
vi.          return Response(status = status.HTTP_404_NOT_FOUND)
vii.
viii.
ix.      if request.method == 'GET':
x.           serializer = StudentSerializer(student)
xi.          return Response(serializer.data)
xii.
xiii.    elif request.method == 'PUT':
xiv.         serializer = StudentSerializer(student, data = request.data)
xv.          if serializer.is_valid():
xvi.             serializer.save()
xvii.            return Response(serializer.data)
xviii.       return Response(serializer.errors, status =
     status.HTTP_400_BAD_REQUEST)
xix.
xx.      elif request.method == 'DELETE':
xxi.         student.delete()
xxii.        return Response(status = status.HTTP_204_NO_CONTENT)
```

6. **Urls.py of fbvApp and fbvSerializers project (Configure and Test) –**

   a. First of all, create urls.py file inside the app directory which is fbvApp and import views of fbvApp there and then add path in urlpatterns, one for GET and POST method and one for primary key method like this.

```
from django.urls import path
from fbvApp import views

urlpatterns = [
    path('students/', views.student_list),
    path('students/<int:pk>', views.student_detail)
]
```

   b. Now open urls.py of project which is fbvSerializers and import app views there and add path for urls.py of fbvApp it should look like this.

```
from django.contrib import admin
from django.urls import path, include
from fbvApp import views

urlpatterns = [
    path('admin/', admin.site.urls),
    path('fbvApp/', include('fbvApp.urls'))
]
```

   c. After this now it's time for testing. To do that first we have to make sure our server is running or not, if it's not running use command "**py manage.py runserver**" . After that visit localhost or http://127.0.0.1:8000/fbvApp/students/ now you can see we got a UI by default by Django and we don't see any data.

   d. To add the data, open Valentina Studio and open **studentdb** then goto SQL Editor and run query **INSERT INTO "fbvApp studentdb" VALUES('1', 'John', '100')** to insert the student data into the database, add two three students there and you can see if its added or not by using **SELECT * FROM "fbvApp_student"**.

   e. Now if we refresh the localhost page, we can see the all the data there and if we add 1 or 2 etc at the end of url we can get that specific students records.

   f. We can also use POSTMAN software to test rest framework.

CLASS BASED VIEW

1. **Create a Project –**
   a. Run this command on terminal "**django-admin startproject cbvSerializers**".
   b. Change directory to that folder "**cd cbvSerializers**" and then use command "**py manage.py startapp cbvApp**".

2. **Model –** Copy the same code of fbvApp.
3. **Migration –** do the same steps which was done with of fbvApp.
4. **Serializer –** Copy the same code of fbvApp.

5. **Implementing Non-Primary key base operations –**
   a. Open views.py of cbvApp and there we will create non primary key based operations in class, basically get and post method. The code will look like this.

```
from django.shortcuts import render
from cbvApp.models import Student
from cbvApp.serializers import StudentSerializer
from rest_framework.response import Response
from rest_framework import status
from rest_framework.views import APIView

# Create your views here.
class StudentList(APIView):
    def get(request):
        students = Student.objects.all()
        serializer = StudentSerializer(students, many=True)
        return Response(serializer.data)

    def post(request):
        serializer = StudentSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data,
status=status.HTTP_201_CREATED)
        return Response(serializer.errors,
status=status.HTTP_400_BAD_REQUEST)
```

b. After that we will create Primary Key based operations in class, basically get, put and delete method. The code will look like this.

```
class StudentDetail(APIView):
    def get_object(self, pk):
        try:
            return Student.objects.get(pk=pk)
        except Student.DoesNotExist:
            return Response(status=status.HTTP_404_NOT_FOUND)

    def get(self, request, pk):
        student = self.get_object(pk)
        serializer = StudentSerializer(student)
        return Response(serializer.data)

    def put(self, request, pk):
        student = self.get_object(pk)
        serializer = StudentSerializer(student, data=request.data)

        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors,
status=status.HTTP_400_BAD_REQUEST)

    def delete(self, request, pk):
        student = self.get_object(pk)
        student.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)
```

6. Configuring URLS and Testing –

a. Go to urls.py of Project cbvSerializers and add the path there for both the class-based views. The code will look like this.

```
from django.contrib import admin
from django.urls import path
from cbvApp import views

urlpatterns = [
    path('admin/', admin.site.urls),
    path('students/', views.StudentList.as_view()),
    path('students/<int:pk>', views.StudentDetail.as_view())
]
```

b. We can visit the localhost in our browser and do the testing now. If you get programming error (data not found), make sure you did the migrations.

MIXINS

- The mixins are a bunch of classes which provide the actions that are used to provide the basic view behavior.
- The mixin classes provide action methods rather than defining the handler methods, such as `.get()` and `.post()`, directly. This allows for more flexible composition of behavior.
- It implements the DRY (Do not repeat yourself) principle.
- ListModelMixin, CreateModelMixin, RetrieveModelMixin, UpdateModelMixin, DestroyModelMixin, these are the types of mixins.

1. **Non-Primary Key Based Operations-**
   a. We will be using cbvSerializer project for this, so open cbvApp views.py and comment all the code. Then we will import generics and mixins from rest_framework and create new class. It will look like this.

```
from gc import get_objects
from django.shortcuts import render
from cbvApp.models import Student
from cbvApp.serializers import StudentSerializer
from rest_framework.response import Response
from rest_framework import status
from rest_framework.views import APIView

from rest_framework import generics, mixins

class StudentList(mixins.ListModelMixin, mixins.CreateModelMixin,
generics.GenericAPIView):
    queryset = Student.objects.all()
    serializer_class = StudentSerializer

    def get(self, request):
        return self.list(request)

    def post(self, request):
        return self.create(request)
```

2. **Primary Key Based Operations-**
   a. Now we will write the code for primary key based operations and it will look like this.

```
class StudentDetail(mixins.RetrieveModelMixin, mixins.UpdateModelMixin,
mixins.DestroyModelMixin, generics.GenericAPIView):
    queryset = Student.objects.all()
    serializer_class= StudentSerializer

    def get(self, request, pk):
        return self.retrieve(request, pk)

    def put(self, request, pk):
        return self.update(request, pk)

    def delete(self, request, pk):
        return self.destroy(request, pk)
```

3. **Configuring URLs and Testing-**
   a. As we changed our old cbvApp we don't have to do anything with the urls. We can directly do the testing.

GENERIC VIEWS

- Generic views are a set of commonly used patterns. They're built on top of the `APIView` class.
- They consist of GenericAPIView, mixins, and concrete views.
    - GenericAPIView is a more loaded version of APIView. It isn't really useful on its own but can be used to create reusable actions.
    - Mixins are bits of common behavior. They're useless without GenericAPIView.
    - Concrete views combine GenericAPIView with the appropriate mixins to create views often used in APIs.
        - CreateAPIView, ListAPIView, RetrieveAPIView, DestroyAPIView, UpdateAPIView, ListCreateAPIView, RetrieveUpdateAPIView, RetrieveDestroyAPIView, RetrieveUpdateDestroyAPIView, these are the types which we can use.
        - ListCreateAPIView can be used for all non-id or non-primary key based operations and RetrieveUpdateDestroyAPIView can be used with all id-base or primary key base operations.

1. **Views.py of cbvApp-**
    a. We will only change the code of views.py of our cbvApp and it will look like this.

```
i.   from courseApp.models import Course
ii.  from courseApp.serializers import CourseSerializer
iii. from rest_framework import generics
iv.  #Generics / Generic Views
v.   class CourseList(generics.ListCreateAPIView):
vi.      queryset = Course.objects.all()
vii.     serializer_class = CourseSerializer
viii.
ix.  class CourseDetail(generics.RetrieveUpdateDestroyAPIView):
x.       queryset = Course.objects.all()
xi.      serializer_class = CourseSerializer
```

2. **Configuring URLs and Testing-**
    a. It’s done we can now test it, there’s no need to do the URL configuration.

VIEW SETS

- ViewSet is a type of class-based view.
- ViewSet is a bunch of view or set of view that we create either by extending the ViewSet class or extending the ModelViewSet class.
- Instead of method handlers, like `.get()` and `.post()`, it provides actions, like `.list()` and `.create()`.
- There are four types of ViewSets, from the most basic to the most powerful-
    - ViewSet
    - GenericViewSet
    - ReadOnlyModelViewSet
    - ModelViewSet

1. **Views.py of cbvApp-**
    a. Go to views.py of cbvApp and import ViewSets from rest_framework there and create a StudentViewSet class, the code will look like this.

```
i.   from cbvApp.models import Student
ii.  from cbvApp.serializers import StudentSerializer
iii. from rest_framework import viewsets
iv.
v.   class StudentViewSet(viewsets.ModelViewSet):
vi.      queryset = Student.objects.all()
vii.     serializer_class = StudentSerializer
```

2. **Configuring URLs and Testing-**
    a. Open urls.py of Project which is cbvSerializers and then setup the router and urlpattern there, the code will look like this.

```
i.   from django.urls import path, include
ii.  from cbvApp import views
iii. from rest_framework.routers import DefaultRouter
iv.
v.   router = DefaultRouter()
vi.  router.register('students', views.StudentViewSet)
vii.
viii. urlpatterns = [
ix.      path('', include(router.urls))
x.   ]
```

    b. Now goto localhost and test it.

NESTED SERIALIZERS

1. **Create a Project –**
   a. Run this command on terminal "**django-admin startproject nestedSerializers**".
   b. Change directory to that folder "**cd nestedSerializers**" and then use command "**py manage.py startapp nsApp**".
   c. Open nestedSerializers in VS Code and open settings.py of project nestedSerializers then configure nsapp and rest framework there.

```
INSTALLED_APPS = [
    'nsApp',
    'rest_framework'
]
```

   d. Then scroll down and configure database.

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'authordb',
        'USER': 'postgres',
        'PASSWORD': 'root',
        'HOST': 'localhost'
    }
}
```

2. **Create the Model –**
   a. Open nsApp models.py and the code should be like this.

```
from django.db import models

class Author(models.Model):
    firstName = models.CharField(max_length=20)
    lastName = models.CharField(max_length=20)

class Book(models.Model):
    title = models.CharField(max_length=20)
    ratings = models.CharField(max_length=10)
    author = models.ForeignKey(Author, related_name='books',
on_delete=models.CASCADE)
```

3. **Migration –**
   a. Before doing the migration, we have to create the database **authordb** which we mentioned in the database section of settings.py.
   b. Open Valentina Studio and create the database named **authordb**.
   c. Use command "**py manage.py makemigrations**" and then "**py manage.py migrate**".

4. **Serializers –**
   a. Goto nsApp and create a file named serializer.py.
   b. Open it and the code should be this.

```
from .models import Author, Book
from rest_framework import serializers
```

```python

class BookSerializer(serializers.ModelSerializer):
    class Meta:
        model = Book
        fields = '__all__'

class AuthorSerializer(serializers.ModelSerializer):
    books = BookSerializer(read_only=True, many=True)
    class Meta:
        model = Author
        fields = '__all__'
```

5. **Views.py of nsApp –**
   a. The code should look like this.

```python
from .models import Author, Book
from nsApp.serializer import AuthorSerializer, BookSerializer
from rest_framework import generics

class AuthorListView(generics.ListCreateAPIView):
    queryset = Author.objects.all()
    serializer_class = AuthorSerializer

class AuthorDetailView(generics.RetrieveUpdateDestroyAPIView):
    queryset = Author.objects.all()
    serializer_class = AuthorSerializer

class BookListView(generics.ListCreateAPIView):
    queryset = Book.objects.all()
    serializer_class = BookSerializer

class BookDetailView(generics.RetrieveUpdateDestroyAPIView):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
```

6. **Configuring URLs and Testing –**
   a. Urls.py of nsApp:

```python
from django.urls import path
from nsApp import views

urlpatterns = [
    path('author/', views.AuthorListView.as_view()),
    path('author/<int:pk>', views.AuthorDetailView.as_view()),
    path('book/', views.BookListView.as_view()),
    path('book/<int:pk>', views.BookListView.as_view()),
]
```

   b. Urls.py of nestedSerializerss:

```python
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('nsApp.urls'))
]
```

   c. Testing – Visit localhost and test it.

PAGINATION

- Django provides a few classes that help you manage paginated data – that is, data that's split across several pages, with "Previous/Next" links.
- REST framework includes support for customizable pagination styles. This allows you to modify how large result sets are split into individual pages of data.

1. **PageNumberPagination (Global Configuration) –**
   a. We will be using cbvSerializer project for this, so open it and go to settings.py of project and add this code there.

```
REST_FRAMEWORK = {
    'DEFAULT_PAGINATION_CLASS':
'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 2
}
```

   b. That's it, use command "py manage.py runserver" to run the server and test it.

2. **PageNumberPagination (Class Level Configuration) –**
   a. Open views.py of cbvApp and create a pagination class, code will look like this.

```
from rest_framework.pagination import PageNumberPagination

class StudentPagination(PageNumberPagination):
    page_size=1;
```

   b. After that in whichever class you like this pagination setting, assign this class name to pagination_class variable like this.

```
class StudentViewSet(viewsets.ModelViewSet):
    queryset = Student.objects.all()
    serializer_class = StudentSerializer
    pagination_class = StudentPagination
```

3. **LimitOffsetPagination (Class Level Configuration) –**
   a. This pagination style mirrors the syntax used when looking up multiple database records. The client includes both a "limit" and an "offset" query parameter. The limit indicates the maximum number of items to return, and is equivalent to the `page_size` in other styles. The offset indicates the starting position of the query in relation to the complete set of unpaginated items.
   b. Open views.py and the code should be like this.

```
from rest_framework.pagination import LimitOffsetPagination

class StudentViewSet(viewsets.ModelViewSet):
    queryset = Student.objects.all()
    serializer_class = StudentSerializer
    pagination_class = LimitOffsetPagination
```

   c. Make sure you added this code to settings.py of project.

```
REST_FRAMEWORK = {
    'DEFAULT_PAGINATION_CLASS':
'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 2
}
```

FILTERING

- The default behavior of REST framework's generic list views is to return the entire queryset for a model manager. Often you will want your API to restrict the items that are returned by the queryset.

1. **Install filter–**
   a. Open command prompt and type “pip3 install django-filter” and it will install the Django-filter in our machine.

2. **Settings.py of cbvSerializers project–**
   a. Open settings.py and add Django filters module in installed app.
   b. This is how it will look.

```
INSTALLED_APPS = [
    'cbvApp',
    'rest_framework',
    'django_filters'
]
```

3. **Views.py of cbvApp–**
   a. Open views.py of cbvApp and import Django filter backends and write the filter code in the class, it will look like this.

```
from cbvApp.models import Student
from cbvApp.serializers import StudentSerializer
from rest_framework import viewsets
from django_filters.rest_framework import DjangoFilterBackend

class StudentViewSet(viewsets.ModelViewSet):
    queryset = Student.objects.all()
    serializer_class = StudentSerializer
    filter_backends = [DjangoFilterBackend]
    filterset_fields = ['name', 'score']
```

   b. Using search filter –

```
from cbvApp.models import Student
from cbvApp.serializers import StudentSerializer
from rest_framework import viewsets
from rest_framework import filters

class StudentViewSet(viewsets.ModelViewSet):
    queryset = Student.objects.all()
    serializer_class = StudentSerializer
    filter_backends = [filters.SearchFilter]
    search_fields = ['id', 'name']
```

   c. There are some extras, we can do with search like if with special symbols like ^ which means starts with, = which means exact match, @ Full-text search. $ Regex search. We have to write these inside the list set of search fields. Like this [‘^id’, ‘=name’]

d. Ordering Filter – We can order the data by default or give the option to user, code should look like this

```
i.   from cbvApp.models import Student
ii.  from cbvApp.serializers import StudentSerializer
iii. from rest_framework import viewsets
iv.  from rest_framework import filters
v.
vi.  class StudentViewSet(viewsets.ModelViewSet):
vii.     queryset = Student.objects.all()
viii.    serializer_class = StudentSerializer
ix.      filter_backends = [filters.OrderingFilter]
x.       ordering_fields = ['name', 'score']
xi.      #always mention the ordering_fields else user can order by sensitive
         fields like passwords
xii.     ordering = ['id'] #default ordering
```

SECURITY

- Authentication is the mechanism of associating an incoming request with a set of identifying credentials, such as the user the request came from, or the token that it was signed with. The permission and throttling policies can then use those credentials to determine if the request should be permitted.
- REST framework provides several authentication schemes out of the box, and also allows you to implement custom schemes.
- Authentication always runs at the very start of the view, before the permission and throttling checks occur, and before any other code is allowed to proceed.

1. Authentication –

a. We will use nestedSerializers project for this and now go to views.py of nsApp and import basicauthentication and isauthenticated module and then assign those two to authentication_classes and permission_classes respectively, the code will look like this.

```
i.   from .models import Author, Book
ii.  from nsApp.serializer import AuthorSerializer, BookSerializer
iii. from rest_framework import generics
iv.  from rest_framework.authentication import BasicAuthentication
v.   from rest_framework.permissions import IsAuthenticated
vi.
vii. class AuthorListView(generics.ListCreateAPIView):
viii.     queryset = Author.objects.all()
ix.       serializer_class = AuthorSerializer
x.        authentication_classes = [BasicAuthentication]
xi.       permission_classes = [IsAuthenticated]
```

b. After that we have to create a superuser to we can login when we visit localhost and view these details, to create superuser type command “py manage.py createsuperuser” and then enter the username and password which you want to choose and its done.

2. Authorization –

a. First of all import djangomodelpermissions to nsApp views like this.

```
i.   from .models import Author, Book
ii.  from nsApp.serializer import AuthorSerializer, BookSerializer
iii. from rest_framework import generics
iv.  from rest_framework.authentication import BasicAuthentication
v.   from rest_framework.permissions import IsAuthenticated,
     DjangoModelPermissions
vi.
vii. class AuthorListView(generics.ListCreateAPIView):
viii.     queryset = Author.objects.all()
ix.       serializer_class = AuthorSerializer
x.        authentication_classes = [BasicAuthentication]
xi.       permission_classes = [IsAuthenticated, DjangoModelPermissions]
```

b. Now we have to create a user through admin panel of Django, to do that visit http://127.0.0.1:8000/admin and login using the superuser credentials we created in authentication part, and create a new user and then go to staff access enable it and then select auth permission of view only access and its done.

3. **Global Security –**
   a. To apply authentication and authorization for whole app or all the classes of views (which means all the endpoints), we need to configure the authentication in projects settings.py.
   b. Open settings.py and add this code.

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES':
['rest_framework.authentication.BasicAuthentication'],
    'DEFAULT_PERMISSION_CLASSES':
['rest_framework.permissions.IsAuthenticated',
'rest_framework.permissions.DjangoModelPermissions']
}
```

   c. We can test by visiting localhost:8000/api/author, Or by using Postman software.

FLIGHT RESERVATION API

- **We will learn some other things like Validations, Token Authentication using this example project.**

1. Create a Project –

a. Run this command on terminal "**django-admin startproject FlightServices**".

b. Change directory to that folder "**cd FlightServices**" and then use command "**py manage.py startapp flightApp**".

c. Open FlightServices in VS Code and open settings.py of project FlightServices then configure flightApp and rest framework there.

```
INSTALLED_APPS = [
    'flightApp',
    'rest_framework'
]
```

d. Then scroll down and configure database.

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'flightdb',
        'USER': 'postgres',
        'PASSWORD': 'root',
        'HOST': 'localhost'
    }
}
```

2. Create the Model –

a. Open flightApp models.py and the code should be like this.

```
from django.db import models

class Flight(models.Model):
    flightNumber = models.CharField(max_length=10)
    operatingAirlines = models.CharField(max_length=20)
    departureCity = models.CharField(max_length=20)
    arrivalCity = models.CharField(max_length=20)
    dateOfDeparture = models.DateField()
    estimatedTimeOfDeparture = models.TimeField()

class Passenger(models.Model):
    firstName = models.CharField(max_length=20)
    lastName = models.CharField(max_length=20)
    middleName = models.CharField(max_length=20)
    email = models.CharField(max_length=30)
    phone = models.CharField(max_length=13)

class Reservation(models.Model):
    flight = models.OneToOneField(Flight, on_delete=models.CASCADE)
    passenger = models.OneToOneField(Passenger,
on_delete=models.CASCADE)
```

3. **Serializers –**
   a. Goto flightApp and create a file named serializer.py, and the code should be like this.

```
from rest_framework import serializers
from flightApp.models import Flight, Passenger, Reservation

class FlightSerializer(serializers.ModelSerializer):
    class Meta:
        model = Flight
        fields = '__all__'

class PassengerSerializer(serializers.ModelSerializer):
    class Meta:
        model = Passenger
        fields = '__all__'

class ReservationSerializer(serializers.ModelSerializer):
    class Meta:
        model = Reservation
        fields = '__all__'
```

4. **Viewsets (Views.py) –**
   a. Open views.py and the code should be like this.

```
from flightApp.models import Flight, Passenger, Reservation
from flightApp.serializers import FlightSerializer, PassengerSerializer,
ReservationSerializer
from rest_framework import viewsets

class FlightViewSet(viewsets.ModelViewSet):
    queryset = Flight.objects.all()
    serializer_class = FlightSerializer

class PassengerViewSet(viewsets.ModelViewSet):
    queryset = Passenger.objects.all()
    serializer_class = PassengerSerializer

class ReservationViewSet(viewsets.ModelViewSet):
    queryset = Reservation.objects.all()
    serializer_class = ReservationSerializer

```

5. **Configuring Router –**
   a. Goto urls.py and the code should look like this.

```
from django.contrib import admin
from django.urls import path, include
from flightApp import views
from rest_framework.routers import DefaultRouter

router = DefaultRouter()
router.register('flights', views.FlightViewSet)
router.register('passengers', views.PassengerViewSet)
router.register('reservations', views.ReservationViewSet)

urlpatterns = [
    path('admin/', admin.site.urls),
    path('flightApi/', include(router.urls))
]

```

6. **Migration –**
   a. Before doing the migration, we have to create the database **flightdb** which we mentioned in the database section of settings.py.
   b. Open Valentina Studio and create the database named **flightdb**.
   c. Use command "**py manage.py makemigrations**" and then "**py manage.py migrate**".

7. **Implementation of findFlights and saveReservation –**
   a. Go to views.py and the code should be like this. And change reservation class flight = models.OneToOneField to models.ForeignKey in models.py file.

```
from flightApp.models import Flight, Passenger, Reservation
from flightApp.serializers import FlightSerializer, PassengerSerializer, ReservationSerializer
from rest_framework import viewsets
from rest_framework.response import Response
from rest_framework.decorators import api_view
from rest_framework import status

@api_view(['POST'])
def find_flights(request):
    flights = Flight.objects.filter(departureCity=request.data['departureCity'], arrivalCity=request.data['arrivalCity'], dateOfDeparture=request.data['dateOfDeparture'])
    serializer = FlightSerializer(flights, many=True)
    return Response(serializer.data)

@api_view(['POST'])
def save_reservation(request):
    flight = Flight.objects.get(id=request.data['flightId'])

    passenger = Passenger()
    passenger.firstName = request.data['firstName']
    passenger.lastName = request.data['lastName']
    passenger.middleName = request.data['middleName']
    passenger.email = request.data['email']
    passenger.phone = request.data['phone']
    passenger.save()

    reservation = Reservation()
    reservation.flight = flight
    reservation.passenger = passenger

    reservation.save()
    return Response(status=status.HTTP_201_CREATED)

class FlightViewSet(viewsets.ModelViewSet):
    queryset = Flight.objects.all()
    serializer_class = FlightSerializer

class PassengerViewSet(viewsets.ModelViewSet):
    queryset = Passenger.objects.all()
    serializer_class = PassengerSerializer

class ReservationViewSet(viewsets.ModelViewSet):
    queryset = Reservation.objects.all()
    serializer_class = ReservationSerializer
```

8. **Testing –**
   a. Before running the server make sure you added findFlights and saveReservation paths in urls.py, it will look like this.

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('flightApi/', include(router.urls)),
    path('flightApi/findFlights/', views.find_flights),
    path('flightApi/saveReservation/', views.save_reservation)
]
```

   b. Run the server using "py manage.py runserver" command and check everything is working, we can also use postman for testing.

- ## Validations
  - **Inbuilt Validations–** Django comes with inbuilt validations if we don't pass the data as we mentioned it will give us the error and generate a warning message. These are the two examples-
    - On flightApp models.py we have set the flightNumber length to 10 digits and if we put more than that it will error out and give us a message for what's wrong.
    - If we don't put the value in departureCity column validation will give us the error until and unless we tell it to that we can set it to blank or empty by mention blank=True in parenthesis as we mention the max_length etc. So, it depends on us we can even allow NULL values to by using null=True, when we will not put anything in departure field it will be null by default.
  - **Custom Validations-** To do the custom validation we have to write the code in serializers.py, it will be like this. There is two ways to do that-
    - First one is where we select a specific data which we want to validate and we have to define the function which needs to follow a convention, for example we have to validate the flight number than we will use the validate_flightNumber name function which is exactly the same as the variable which we defined in models.py.
    - When we use the second way we can access and validate the whole data for that we have to use it like dictionary, when we call a value from dictionary using a key, we can do that same here we have to write **data['flightNumber']** to get the flight number.

```
from rest_framework import serializers
from flightApp.models import Flight, Passenger, Reservation
import re

class FlightSerializer(serializers.ModelSerializer):
    class Meta:
        model = Flight
        fields = '__all__'

    def validate_flightNumber(self, flightNumber):
        if(re.match("^[a-zA-Z0-9]*$", flightNumber) == None):
            raise serializers.ValidationError("Invalid Flightnumber,
Make sure it is AlphaNumeric")
        return flightNumber

    def validate(self, data):
        print(data['flightNumber'])
        print(data['departureCity'])
        return data
```

- **Token Based Authentication**
  - **Configuration-**
    - Open settings.py of project FlightServices and add rest auth app in installed apps and then we have to add rest framework block, it will look like this.

```
INSTALLED_APPS = [
    'django.contrib.staticfiles',
    'flightApp',
    'rest_framework',
    'rest_framework.authtoken'
]

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES' : [
        'rest_framework.authentication.TokenAuthentication'
    ]
}
```

    - Open urls.py and add path there it will look like this.

```
from django.contrib import admin
from django.urls import path, include
from flightApp import views
from rest_framework.authtoken.views import obtain_auth_token

urlpatterns = [
    path('admin/', admin.site.urls),
    path('flightApi/', include(router.urls)),
name="api_token_auth")
]
```

    - Open views.py and add permission_classes to all the viewsets which you want to be authenticated. It will look like this. After this step run the migrations.

```
from rest_framework import status
from rest_framework.permissions import IsAuthenticated

class FlightViewSet(viewsets.ModelViewSet):
    queryset = Flight.objects.all()
    serializer_class = FlightSerializer
    permission_classes = (IsAuthenticated,)
```

  - **Create Users & Tokens-**
    - Before creating the users and tokens we have to create the superuser use **"py manage.py createsuperuser"** command to do that.
    - Go to localhost/admin and login using that id pass you just created using superuser command.
    - Click on add user and create a username and password then click on add token and select username and click on save to create the token.

  - **Testing-**
    - Open Postman go to http://127.0.0.1:8000/flightApi/flights/ and when we use GET or POST it will generate the error authentication credentials not provided.
    - To retrieve the token, open new postman tab select method POST and go to url http://127.0.0.1:8000/api-token-auth/ it will generate the warning username and password not provided click on body then raw then select type to json after that write username and password in json format,

which is

```
{
        "username" : "test",
        "password" : "Test#123"
}
```

Then click on send and it will give us the token

- Copy the token and then open previous tab where we were creating the flight and then click on headers and inside the key column write Authorization and on values field after writing Token give a space and paste the token. Now if we click send the new flight will be created.

- **Automate Token Creation-**
  - Go to models.py do the imports and then define the function to do the automatic creation of token when a user is created. The code will look like this.

```
from django.db import models

from django.db.models.signals import post_save
from django.dispatch import receiver
from rest_framework.authtoken.models import Token
from django.conf import settings

@receiver(post_save, sender=settings.AUTH_USER_MODEL)
def createAuthToken(sender, instance, created, **kwargs):
    if created:
        Token.objects.create(user=instance)

class Flight(models.Model):
    flightNumber = models.CharField(max_length=10)
    operatingAirlines = models.CharField(max_length=20)
    departureCity = models.CharField(max_length=20, null=True)
    arrivalCity = models.CharField(max_length=20)
    dateOfDeparture = models.DateField()
    estimatedTimeOfDeparture = models.TimeField()
```

ANGULAR FRONTEND

- First step is to install angular and to do that, We will install node.js visit https://nodejs.org/en/download/ and download and install node.js and after that run "**npm -v**" command to check if it is working or not.
- Now launch command prompt and run **"npm install -g @angular/cli"** to install angular.
- After the installation we can create new app using **"ng new my-app"** command here my-app is the app name which can be anything.
- Then use **"cd my-app"** to enter that directory and use "ng serve –open" command to run the app it will be launched and the url will be **"localhost:4200"** or **"127.0.0.1:4200"**.
- Open models.py of flightApp and make sure you used ForeignKey, if we don't we will not be able to repeat same flights and same passengers.

```
class Reservation(models.Model):
    flight = models.ForeignKey(Flight, on_delete=models.CASCADE)
    passenger = models.ForeignKey(Passenger, on_delete=models.CASCADE)
```

- Now we will be running django server on 127.0.0.1:8000 and angular server on 127.0.0.1:4200 and browser will not allow both servers to communicate with each other until we use some middleware, so we will install Django cors headers.

- Use **“pip install django-cors-headers”** command to install it, then goto settings.py of project FlightServices and in INSTALLED_APPS block add ‘corsheaders’. It will look like this.

```
INSTALLED_APPS = [
    'flightApp',
    'rest_framework',
    'rest_framework.authtoken'
    'corsheaders'
]
```

- Scroll down and in the MIDDLEWARE block add **“corsheaders.middleware.CorsMiddleware”** and create new code block inside the settings **ALLOWED_HOSTS =[*]** and another one **CORS_ORIGIN_ALLOW_ALL=True**. This will allow any application running on any port to access Django server. If you want to allow only single application to access Django server you can use.

```
#cors config
CORS_ORIGIN_ALLOW_ALL = False

CORS_ORIGIN_WHITELIST = (
    'http://localhost:4200',
)
```

- Create new project named flightReservation using **“ng new flightReservation”** command, after that change directory into that folder and now we have to create four components and for that we will use **“ng g c components/findFlights”** command like this create **“ng g c components/displayFlights” , “ng g c components/passengerDetails”** and **“ng g c components/confirm”** to confirm the reservation.
- Now we have to create login and reservation services and to do that use **“ng g s services/login”** and **“ng g s services/reservation”** command respectively.